

Week 1 Reflection: Multi-Agent Research & Report Crew

Course: UCLA Extension — Agentic AI

Author: Bhavesh Upadhyaya

Repository: github.com/bhaveshu/agents-assignment-1

1. Architectural & Agent Design: The "Why"

For this assignment, I implemented a 4-agent sequential workflow in CrewAI designed to produce comprehensive academic literature reviews from a curated 15-paper corpus. The core architectural philosophy was **strict separation of concerns** across four specialized personas matching the exact code implementation:

1. **query_expander (Role: Research Query Strategist)**: Focuses solely on conceptual problem decomposition. It translates ambiguous, high-level research questions into 3–5 targeted sub-questions, academic keywords, and search queries.
2. **source_hunter (Role: Investigative Source Hunter)**: The dedicated RAG specialist. It is the **only agent equipped with tools** (`search_papers`). By isolating tool execution to this single agent, the pipeline avoids tool-calling confusion and hallucinations in downstream reasoning agents.
3. **synthesizer (Role: Synthesizer – puts it all together)**: Rather than summarizing isolated papers, this agent acts as an analytical synthesizer—detecting points of consensus, theoretical debates, trade-offs, and research gaps across retrieved sources.
4. **report_writer (Role: Lead Academic Technical Writer)**: Specializes exclusively in narrative structure, academic tone, LaTeX equation rendering, and citation formatting in Markdown.

Design Trade-offs & Process Selection

- **Sequential vs. Hierarchical Crew**: I selected `Process.sequential` over a hierarchical architecture with a manager agent. Academic literature review has a strict linear dependency: one cannot search without expanded queries, cannot synthesize without sources, and cannot write without synthesis. A manager agent would have added unnecessary token overhead, latency, and orchestration complexity.
 - **Explicit Context Chaining vs. Built-in Memory**: Rather than relying on CrewAI's automatic memory subsystem, I configured `memory=False` across the Crew and agents, using explicit task context dependencies (`context=[expand_task, search_task, ...]`). This eliminated memory-bus failures while ensuring downstream agents receive 100% full-text fidelity of previous findings.
-

2. The ReAct Paradigm in Action

The execution traces clearly demonstrated the **ReAct (Reasoning + Acting)** pattern, specifically within the **Investigative Source Hunter** agent. Rather than performing a single naive vector search, the agent engaged in an iterative **Thought** → **Action** → **Observation** loop:

- **Thought:** *"I need to discover evidence on how multi-agent systems coordinate and prevent communicative deadlocks. First, I should search for CAMEL and communication protocols."*
- **Action:** `search_papers(query="CAMEL communicative agents role playing inception prompting", k=5)`
- **Observation:** The tool returned relevant excerpts from `08_camel.pdf` explaining inception prompting and role flipping.
- **Thought:** *"The CAMEL paper provides cooperative dialogue evidence, but I still lack evidence on how conversational frameworks handle human-in-the-loop oversight and code execution. I need to search AutoGen next."*
- **Action:** `search_papers(query="AutoGen multi-agent conversation UserProxyAgent execution", k=5)`
- **Observation:** The tool returned passages from `10_autogen.pdf` highlighting `UserProxyAgent` safety modes and Docker sandboxing.

This interleaved loop prevented open-loop hallucination: each subsequent search query was grounded in what the agent had already observed, mimicking how human researchers systematically navigate literature.

3. Debugging Journey & Key Technical Learnings

Iterating on this pipeline surfaced critical systems-engineering lessons:

1. The Citation Hallucination Trap & RAG Metadata Grounding:

This was the most humbling and educational moment of the assignment. When my initial reports were generated, the citations looked deceptively authoritative at first glance—the formatting was crisp, dates matched, and the prose read like a legitimate academic paper. However, grading feedback revealed that the author lists were completely fabricated (e.g., the LLM invented co-authors for Toolformer) and benchmark numbers had drifted (e.g., misstating ALFWorld baseline margins).

Tracing the root cause showed that `tools/paper_rag_tool.py` only returned raw text chunks and section headers; it never fed author metadata to the agents. When the prompt instructed the report writer to include academic citations, the LLM did what autoregressive models do: it hallucinated plausible-sounding co-authors out of thin air.

I resolved this through a defense-in-depth fix:

- **Payload Enrichment:** I updated `tools/paper_rag_tool.py` to load and cache `data/papers/paper_index.json`, explicitly injecting `Authors: ...` and `Year: ...` into every retrieved passage.
- **Prompt Grounding Constraints:** In `tasks/task_definitions.py`, I instructed the writer to use *only* the retrieved author metadata and fact-check all numerical metrics directly against the excerpts.
Re-running the reports with this metadata eliminated the hallucinations completely, demonstrating that agents cannot cite what they cannot see.

2. Resolving Code vs. Reflection Discrepancies (`memory=False` & Model Routing):

In my initial submission, my reflection draft described an architecture that diverged from my committed code: the reflection stated `memory=False`, yet `crew.py` still had `memory=True`

committed, and I had referenced an `llm=` parameter that wasn't explicitly in the agent constructors. This occurred during rapid troubleshooting iterations between parameter testing and environment configs.

To resolve this deterministically:

- **Model Configuration:** Rather than hardcoding an `llm=` parameter into each agent, LLM routing is managed cleanly at the environment level via `OPENAI_MODEL_NAME=gemini/gemini-3.8-flash` and `OPENAI_API_BASE=https://generativelanguage.googleapis.com/v1beta/openai/` in `.env`, loaded via `load_dotenv(override=True)`. CrewAI resolves this natively across all agents.
- **Disabling Memory:** In `crew.py`, I explicitly set `memory=False`. CrewAI's built-in memory attempts to communicate with external embedding endpoints, causing intermittent `memory_save_failed` bus warnings. Setting `memory=False` eliminated these errors while preserving complete context via task context chaining (`context=[expand_task, search_task, ...]`).

3. Embedding Dimension Mismatch (3072 vs. 768):

The initial verification script failed during embedding validation. The starter kit hardcoded Gemini embeddings to 768 dimensions (the legacy standard), whereas Google's modern `gemini-embedding-001` defaults to **3072 dimensions** via Matryoshka Representation Learning. Because ChromaDB was indexed with 3072-dimensional vectors from the PDFs, aligning `dimension = 3072` in `tools/embeddings.py` resolved the mismatch and enabled high-precision retrieval.

4. Starter Kit `.gitignore` Trap:

The starter kit originally included `outputs/*` in `.gitignore`, which quietly prevented generated literature reviews from being tracked by git. Removing `outputs/*` from `.gitignore` ensured all generated reports and evaluation summaries were properly tracked and merged.

5. Codebase Hygiene:

Cleaned up dead code (including an orphaned `NotImplementedError` placed after a `return` statement in `tasks/task_definitions.py`), removed obsolete `# TODO:` comments, and deleted the untracked crash log `error.txt`.

4. Production Considerations

To evolve this prototype into an enterprise-grade research platform:

1. **Dynamic & Multi-Modal Ingestion:** Extending the RAG pipeline beyond static local PDFs to query live arXiv, Semantic Scholar, and web-crawling endpoints for real-time literature updates.
2. **Asynchronous Parallel Retrieval:** The current sequential execution runs tool queries one by one. In production, the `Investigative Source Hunter` should execute multi-query searches concurrently using `asyncio` to reduce retrieval latency.
3. **Automated Citation & Fact Verification:** Incorporating a dedicated critic agent to run formal NLI (Natural Language Inference) checks comparing generated assertions against retrieved chunk IDs before finalizing reports.
4. **Token Economics & Rate Limiting:** Enforcing token budgets and exponential backoff retry wrappers on external model calls to guarantee API resilience.